SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

LoCor AI, Business Intelligence & Market Analysis Agent

Domain: AI for Economic Empowerment & Decision Intelligence

Field	Details
Team Name	Team 42 + 2
Team Members	Gilbert Gan Wei Xuan Colin Leong Hong Seng Gan Jayci Ow Ji Joe Tan Wen Zhe
Institution	Sunway University
Version	2.0
Date	26th of April 2026

Contents

Introduction 4
Purpose 4
Key Elements Covered 4
Background 4
Previous Version 4
Major Architectural Capabilities Introduced in Version 1.0 5
Target Stakeholders 5
System Architecture & Design 6
High Level Architecture 6
LLM as a Service Layer 7
Dependency Diagram 7
Sequence Diagram, Chat with RAG Flow 8
Sequence Diagram, Pricing Strategy Flow 9
Technological Stack 9
Key Data Flows 11
Data Flow Diagram 11
Normalised Database Schema 12
SQLite, chat_messages 12
ChromaDB, Collections 12
Redis, Pricing Cache 13
Functional Requirements & Scope 14
Minimum Viable Product 14
Non-Functional Requirements (NFRs) 14
Out of Scope / Future Enhancements 15
Monitor, Evaluation, Assumptions & Dependencies 16
Technical Evaluation 16
Grayscale Rollout & A/B Testing 16
Emergency Rollback & Golden Release 16
Priority Matrix 16
Monitoring 17
Assumptions 17
External Dependencies 17
Project Management & Team Contributions 19
Project Timeline 19
Team Member Roles 20

Recommendations.....21
References & Citations 22
AI Tools Used 22
AI Usage 23

Introduction

Purpose

This System Analysis Document defines the technical scope and design decisions behind LoCor AI, an AI-powered business intelligence platform for SMEs. It covers the system architecture, data flows, AI integration model, functional scope, non-functional requirements and project management considerations. This document serves as the primary technical reference for all developers and testers working on the platform.

Key Elements Covered

- Architecture: A client-server platform with a FastAPI backend, React frontend, ChromaDB vector store, SQLite chat history, Redis cache and Z.ai GLM-4.5-Flash as the AI reasoning layer.
- Data Flows: How uploaded business documents move from the user's browser through parsing, vectorisation and retrieval into AI-generated outputs.
- Model Process: End-to-end workflows for each of the five core features, upload, chat, dashboard/insights, pricing and report generation.
- Role of Reference: This document is the authoritative technical reference for all team members.

Background

SME business owners operating in retail and e-commerce maintain their business data across disconnected spreadsheets and documents. Extracting strategic insight from this data requires either manual analysis (time-intensive) or a business analyst (cost-prohibitive). LoCor AI was designed to bridge this gap by acting as an AI-powered layer on top of data the owner already has.

Previous Version

LoCor AI is a greenfield product (Version 1.0). There is no prior production system. The design was informed by the limitations observed in generic AI chatbots: they lack access to the user's own business data, cannot produce domain-specific outputs (inventory reports, pricing strategy) and do not integrate with business document formats (.docx, .xlsx).

Major Architectural Capabilities Introduced in Version 1.0

- RAG architecture with ChromaDB [2]: Business data is vectorised on upload and semantically retrieved per query, grounding all AI outputs in actual user data.
- Streaming AI responses: Both chat and pricing strategy endpoints use Server-Sent Events (SSE) to stream tokens in real time.
- Automated document generation: AI-authored content is converted to production-ready .docx and .xlsx files programmatically.
- Live competitor data integration: The Lazada API is integrated as a real-time data source for pricing intelligence.
- Token optimisation via TOON encoding: Competitor data is encoded in a more compact serialisation format before being sent to the model.

Target Stakeholders

Stakeholder	Role	Expectations
SME Business Owner	Primary user. Uploads data, views dashboard, receives weekly reports, asks AI questions.	Simple upload flow, clear KPI summaries, actionable recommendations in plain language.
Operations / Inventory Manager	Uses insights and chat to monitor stock levels and category performance.	Accurate inventory status flags, specific restock and discontinue recommendations.
E-commerce Seller	Uses pricing strategy feature for competitive positioning.	Exact RM price recommendations backed by live competitor data, supply gap identification.
Development Team	Builds and maintains the platform.	Clear API contracts, modular codebase, well-documented architecture.
QA Team	Validates system behaviour across all features.	Defined acceptance criteria, request/response format specs, edge case coverage.

System Architecture & Design

High Level Architecture

Type	Detail
System	Web Application (Browser-based, React 19 frontend)
Architecture	Client-Server with a monolithic FastAPI backend, persistent vector store, SQL store and in-memory cache
Deployment	Local development server. Redis on localhost:6379. ChromaDB on local disk at ./chroma_db.

LoCor AI is structured as a client-server system with a single-page React frontend and a FastAPI Python backend. The frontend communicates with the backend via a REST API and SSE streaming. The backend coordinates five subsystems: DocumentParser (ingestion), VectorStore/VectorRetriever (RAG), AIChat/AIReportGenerator (model orchestration), CacheManager (Redis) and the processing generation layer (generate.py, newsletter.py). All five subsystems interact with the Z.ai GLM-4.5-Flash [1] model as an external service.

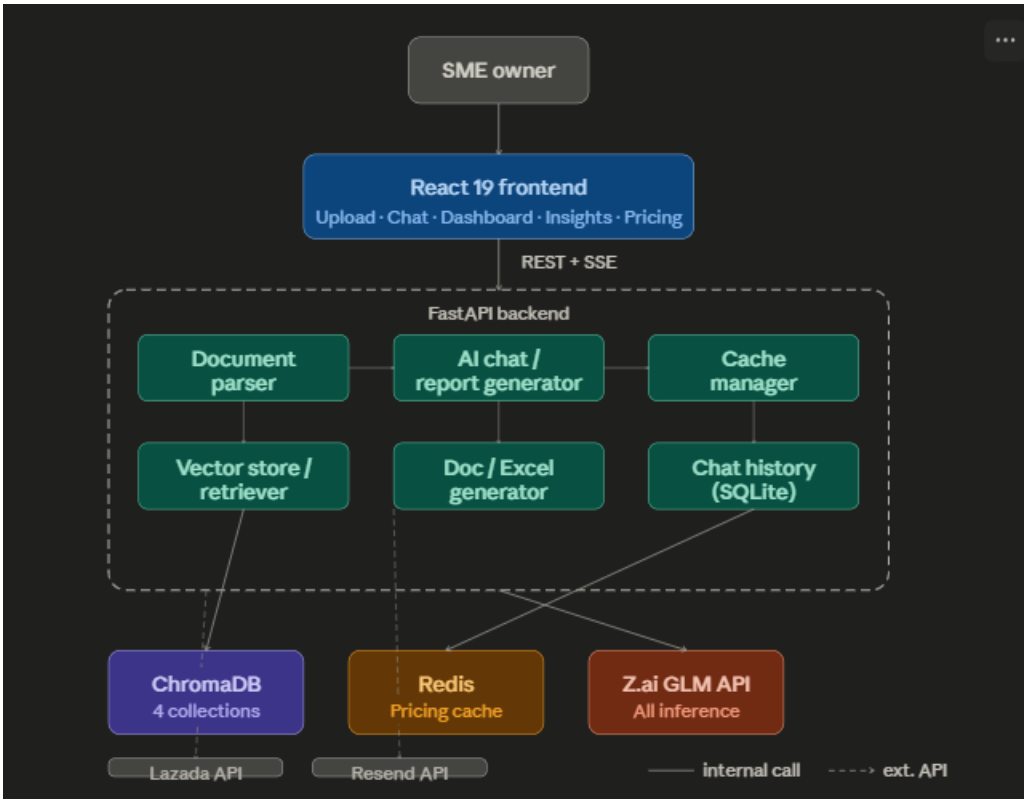

Fig. 1. High Level Architecture

LLM as a Service Layer

Dependency Diagram

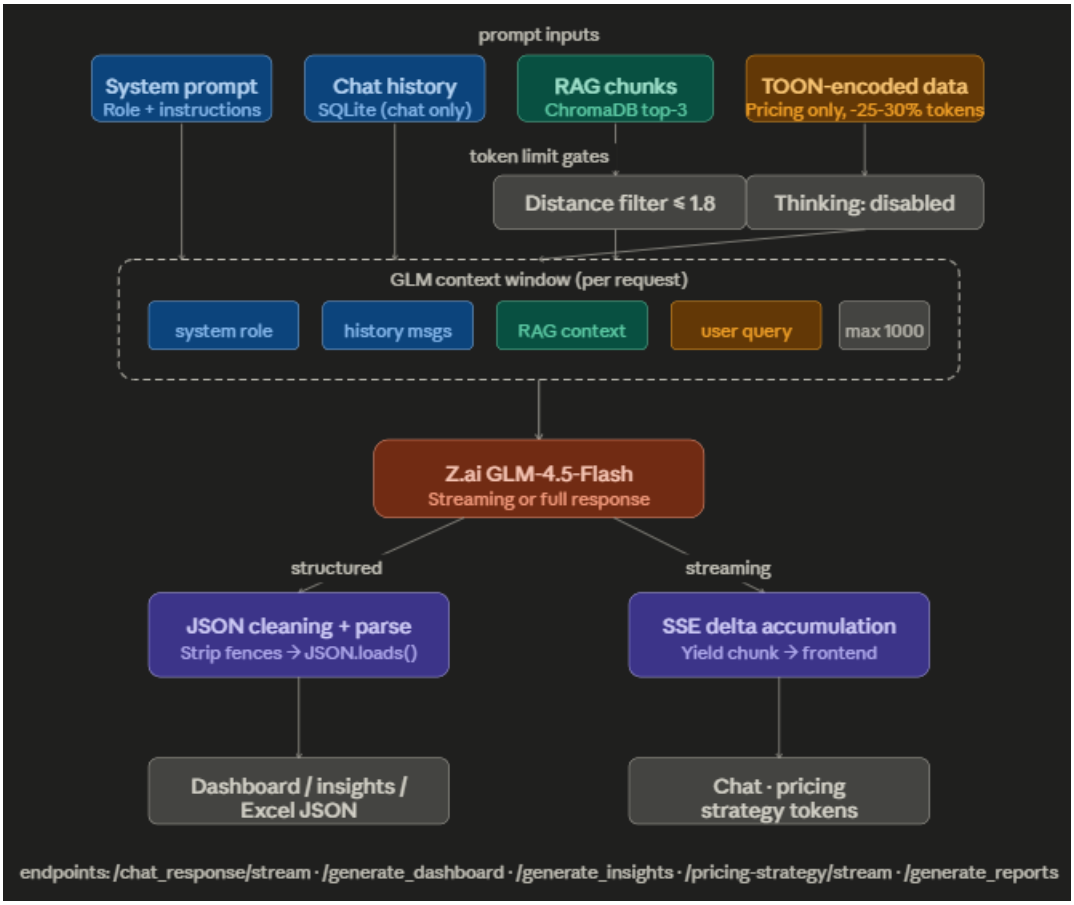

Fig. 2. LLM Dependency / Prompt Construction

The following table describes all major API calls between system components:

From	To	Call / Event	Notes
React Frontend	FastAPI /initialise_data	POST multipart/form-data (4 files)	Triggers DocumentParser + VectorStore
React Frontend	FastAPI /chat_response/stream	POST JSON; SSE response	Chat message + streaming AI reply
React Frontend	FastAPI /generate_dashboard	GET; JSON response	Triggers RAG + GLM; returns dashboard JSON
React Frontend	FastAPI /generate_insights	GET; JSON response	Triggers RAG + GLM; returns insights JSON
React Frontend	FastAPI /pricing-strategy/stream	POST JSON; SSE response	Product name → pricing analysis stream

React Frontend	FastAPI /generate_reports	GET; JSON status	Triggers doc + Excel generation + email
React Frontend	FastAPI /products	GET; JSON response	Returns product list with computed margin
FastAPI	ChromaDB [2] (local)	Python client: add / query	VectorStore.add(), VectorRetriever.query()
FastAPI	SQLite (local)	sql.get_all_messages / save_message	Chat history persistence
FastAPI	Redis [3] (localhost:6379)	CacheManager.serve_cache / store_cache	Pricing result caching per product name
FastAPI	Z.ai GLM API [1]	HTTP POST /v1/chat/completions (streaming + non-streaming)	All AI inference calls
FastAPI	Lazada API	HTTP GET (via fetch_all_apis.py)	Live competitor product data
FastAPI	Resend API	HTTP POST /emails	Report + spreadsheet email delivery

Sequence Diagram, Chat with RAG Flow

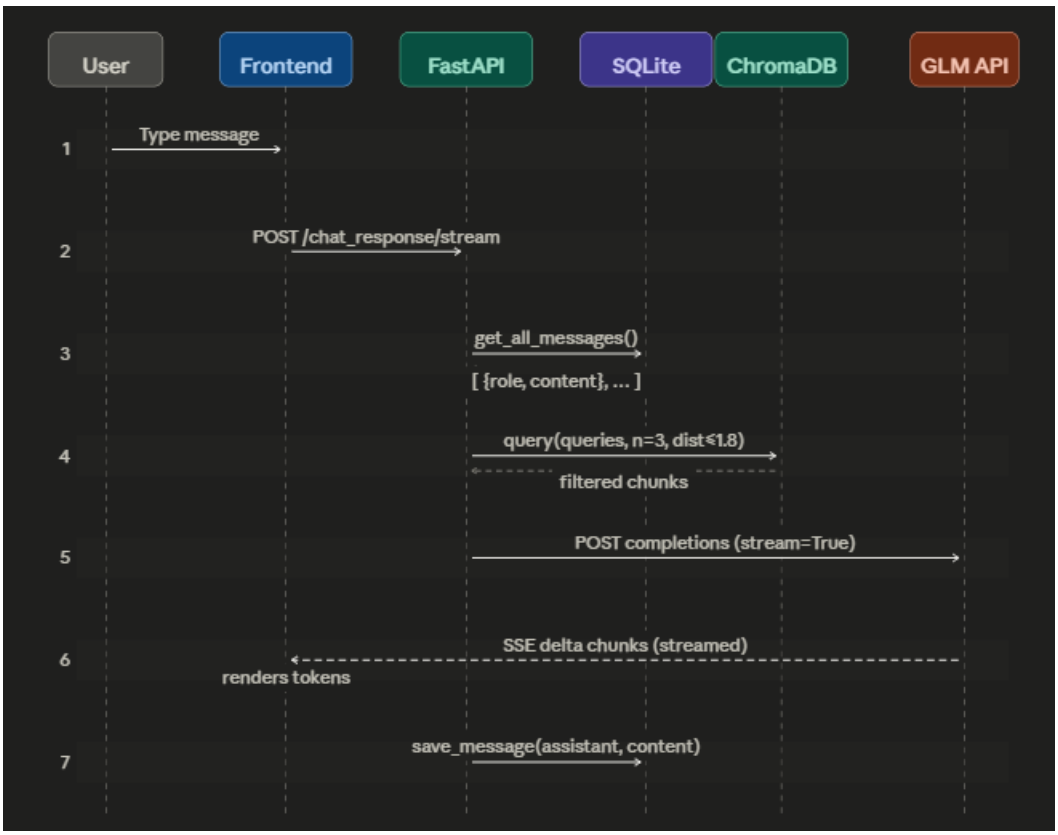

Fig. 3. Sequence Diagram — Chat with RAG Flow

Sequence Diagram, Pricing Strategy Flow

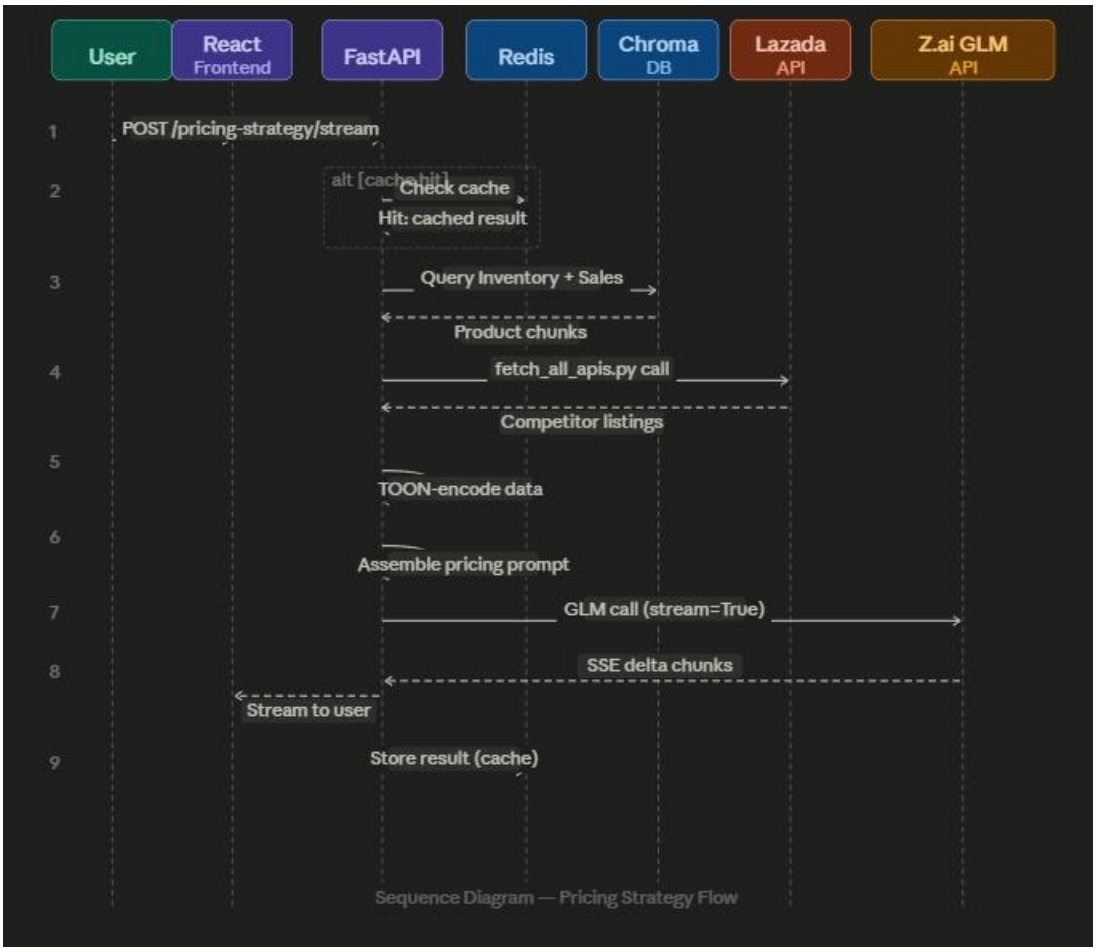

Fig. 4. Sequence Diagram — Pricing Strategy Flow

Technological Stack

Layer	Technology	Justification
Frontend Framework	React 19 (Vite)	Fast build tooling, modern hooks-based architecture, large ecosystem.
Frontend Routing	React Router 7	Declarative routing for the multi-page SPA.
Frontend Styling	TailwindCSS 4	Utility-first CSS for rapid UI development without a component library overhead.
Backend API	FastAPI [4] (Python 3.10+)	Async-first, high-performance, built-in OpenAPI docs, native SSE support via StreamingResponse.

AI Model	Z.ai GLM-4.5-Flash [1]	Flash-class speed optimised for streaming; lower cost than full-scale models; reliable JSON output.
Vector Database	ChromaDB [2] (PersistentClient)	Local-first, no external service required; supports metadata filtering and cosine similarity.
SQL Database	SQLite	Zero-config, file-based; appropriate for single-user chat history storage.
Cache	Redis	In-memory key-value store; sub-millisecond cache reads for pricing results.
Document Parsing	PyMuPDF [8] (fitz), python-docx [9]	Reliable PDF and DOCX text extraction without cloud dependencies.
Data Processing	pandas [13], NumPy	Standard data science stack for spreadsheet ingestion and DataFrame manipulation.
Document Generation	python-docx [9], openpyxl	Programmatic .docx and .xlsx generation with conditional formatting support.
Email Delivery	Resend API	Developer-friendly transactional email API with attachment support.
Token Optimisation	json-toon	Custom compact serialisation format; reduces structured data token count by 25-30%.

Key Data Flows

Data Flow Diagram

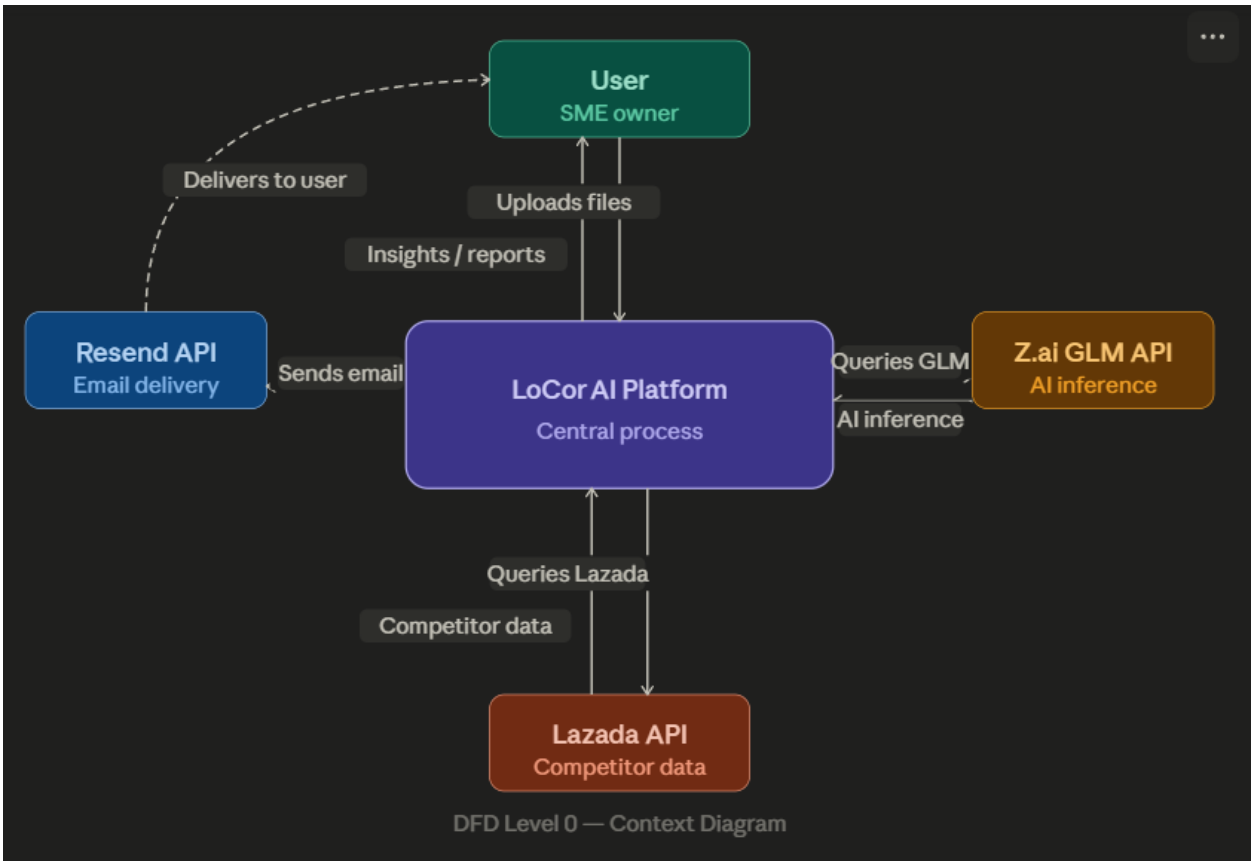

Fig. 5. Data Flow Diagram — Level 0 Context

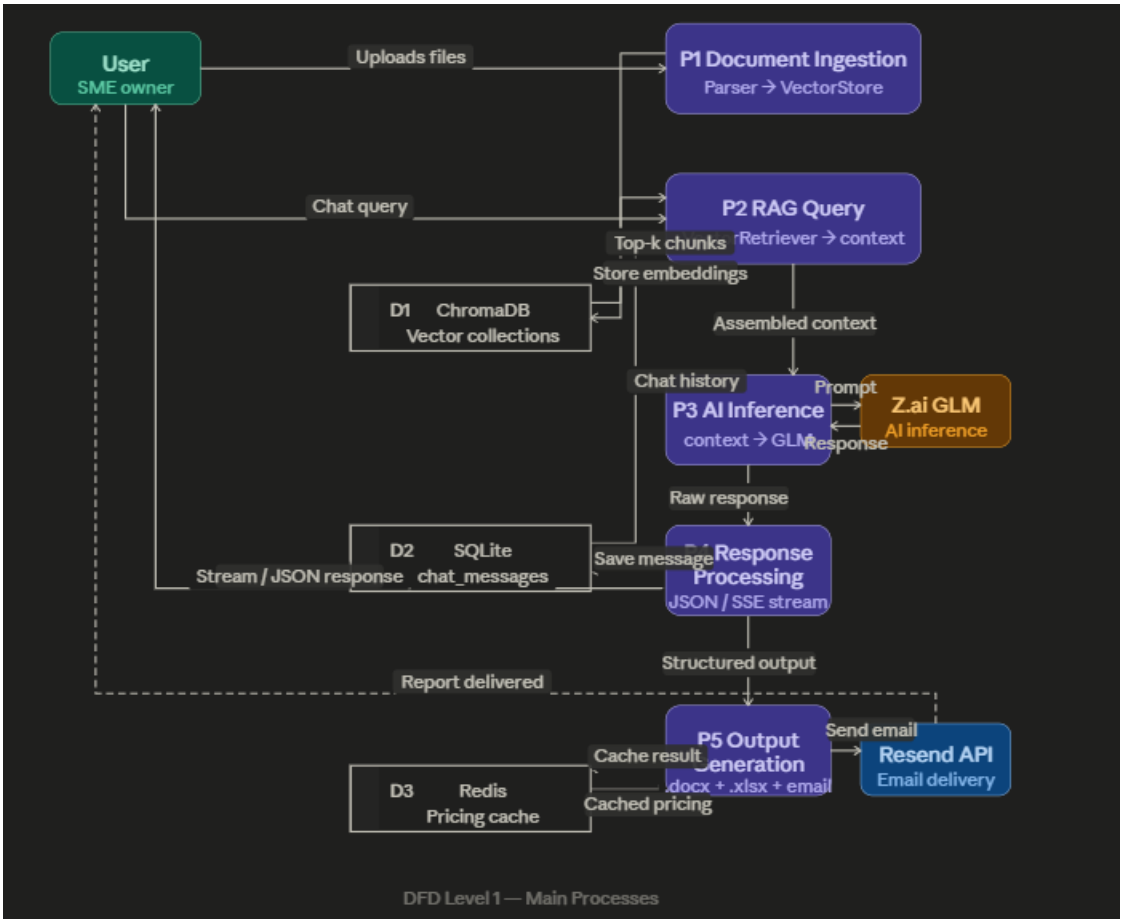

Fig. 6. Data Flow Diagram — Level 1 Main Processes

Normalised Database Schema

SQLite, chat_messages

Column	Type	Constraints	Notes
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique message identifier
role	TEXT	NOT NULL	'user' or 'assistant'
content	TEXT	NOT NULL	Full message text
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Message timestamp

ChromaDB, Collections

ChromaDB does not use a relational schema. Each of the four collections (Company_Description, Inventory_Sheets, Sales_Sheets, Balance_Sheets) stores documents with the following structure:

Field	Type	Notes
id	string (UUID)	Unique document chunk identifier, generated at ingest
document	string	Raw text content of the chunk (company description text or pandas [13] DataFrame repr)
metadata.date_added	string (DD/MM/YYYY)	Date the document was ingested
embedding	float[]	Vector embedding generated by ChromaDB's default embedding model

Redis, Pricing Cache

Pricing strategy results are cached as plain string key-value pairs. Key: product name (lowercased). Value: complete streaming response text. TTL: Not currently set (persistent until Redis restart or manual eviction). Future improvement: add a 24-hour TTL to ensure pricing data reflects current market conditions.

Functional Requirements & Scope

Minimum Viable Product

#	Feature	Description
1	Data Upload & Vectorisation	User uploads four business documents (company description + three spreadsheets). System parses, chunks and stores all data in ChromaDB. Returns success status.
2	AI Chat with RAG	User asks natural-language business questions. System retrieves relevant data chunks and streams a grounded response. Chat history persists across sessions.
3	Dashboard & Insights Generation	System generates a structured JSON dashboard with KPIs, category breakdown, top products and trend analysis. Separate insights endpoint provides action-oriented recommendations per category.
4	Competitive Pricing Strategy	User enters a product name. System fetches live Lazada competitor data, retrieves own product data and streams a structured pricing analysis with an exact RM recommendation.
5	Automated Report Generation & Email	System generates a Word business proposal and an Excel inventory update, then emails both as attachments to the configured recipient.

Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	System must handle typical single-user session load without degradation. Concurrent user support is out of scope for MVP.	FastAPI [4] async architecture handles I/O-bound operations (GLM API, Lazada API) without blocking. Redis reduces Lazada API call frequency.
Reliability	Chat and upload endpoints must not crash on malformed inputs. Pricing endpoint must degrade gracefully on Lazada API failure.	DocumentParser returns None on unsupported file types rather than raising. Pricing endpoint returns a graceful error stream if all APIs fail. SQLite saves chat messages only on stream completion to avoid partial saves.

Maintainability	Each functional area (chat, report, parsing, retrieval, caching) is separated into its own module/class.	ai_generation/, processing_tools/, processing_generation/, vector_db/, chat_history/, apis/ are all independent modules. Prompts are isolated in prompts/ for easy iteration.
Token Latency	Chat and pricing stream first-token latency must be under 5,000ms under normal load. Dashboard/insights response must complete within 10,000ms.	Thinking mode disabled on all GLM calls. TOON encoding reduces pricing prompt size by 25-30%. ChromaDB distance threshold prevents irrelevant chunks from inflating context.
Cost Efficiency	Average token cost per heavy user session must remain within a defined budget threshold.	TOON encoding, Redis caching, thinking-disabled mode and GLM-4.5-Flash (vs full-scale model) collectively reduce per-session token cost by an estimated 35-40% compared to a naive implementation.
Streaming UX	Users must receive visual feedback within 3 seconds of initiating a chat or pricing request.	SSE streaming begins as soon as the first token is available. FastAPI's StreamingResponse yields each delta chunk with asyncio.sleep(0) to avoid blocking the event loop.

Out of Scope / Future Enhancements

- Multi-user support with per-user data isolation and authentication.
- Scheduled weekly report delivery (cron-based automation).
- Direct integration with POS systems, Shopify or ERP platforms.
- Competitor data from Shopee, Amazon or other platforms.
- In-app report viewer (reports are email-only in V1).
- Mobile-responsive frontend.
- Cloud deployment (AWS, GCP or Azure) with managed ChromaDB and Redis.
- ChromaDB document versioning to prevent stale data accumulation across re-uploads.

Monitor, Evaluation, Assumptions & Dependencies

Technical Evaluation

Grayscale Rollout & A/B Testing

For the MVP phase (single-user, local deployment), formal grayscale rollout is not applicable. For future multi-user deployment: new AI prompt versions would be released to 5% of sessions first, with output quality monitored via the AI Output Pass Rate metric (target: >=80%). If quality holds across 100 sessions, the prompt is promoted to 100% of traffic. A/B testing would compare prompt variants by measuring user-perceived response quality via a thumbs up/down UI element.

Emergency Rollback & Golden Release

For the current local deployment: the previous stable version is maintained as a tagged Git release. If a new deployment introduces regressions (as detected by the QA test suite), the team reverts to the previous tag. For future cloud deployment: a blue/green deployment strategy is recommended, with the previous version remaining live until the new version passes all CI/CD gates.

Priority Matrix

Priority	Trigger Condition	Action
P0, Critical	Any upload or chat endpoint returns 500 for all requests; SQLite corruption; ChromaDB collection unqueryable.	Immediate rollback to previous release. Incident logged. Root cause analysis before re-deploy.
P1, High	Pricing strategy endpoint fails for >50% of product queries; AI Output Pass Rate drops below 70%; Redis connection lost.	Hotfix deployed within 4 hours. Mock pricing mode activated as interim mitigation.
P2, Medium	Dashboard/insights JSON parse fails for >20% of requests; email delivery failure rate >10%.	Fix within next sprint. Monitoring alert configured. Manual workaround documented.
P3, Low	Individual edge case failures (unsupported file type, empty dataset); UI rendering glitches.	Logged in backlog. Addressed in next planned release.

Monitoring

The following monitoring plan covers the MVP deployment:

- Server logs: FastAPI [4] stdout logs capture all endpoint calls, ChromaDB [2] query results, Redis [3] cache hits/misses, Lazada API responses and GLM response times. Logs are the primary diagnostic tool.
- Health checks: Redis connectivity is verified by the CacheManager on each pricing request. A startup health check for Redis is recommended as a future improvement.
- AI output quality: The QA team runs the AI Output Test Pairs (Section 6.1 of QATD) after each significant prompt change to verify pass rate remains above 80%.
- Error rate monitoring: For future cloud deployment, AWS CloudWatch or Datadog would be configured to alert if 5xx error rate exceeds 1% over a 5-minute window.

Assumptions

- Users have a stable internet connection during upload and AI generation operations.
- The Z.ai GLM-4.5-Flash [1] API is available and within rate limits during normal operation.
- Redis [3] is running on localhost:6379 before the FastAPI server is started.
- The Resend API [11] key is valid and the recipient email address is configured correctly in .env.
- Uploaded spreadsheets follow a consistent column naming convention compatible with the pandas repr parser (e.g. columns named 'product', 'stock', 'unit price (rm)', 'quantity sold').
- The business operates in Malaysian Ringgit (RM); all monetary formatting assumes RM denomination.

External Dependencies

Tool / Service	Purpose	Risk Level	Mitigation
Z.ai GLM-4.5-Flash [1] API	Core LLM for all inference: chat, insights, dashboard, pricing, report generation.	High, if GLM API is rate-limited or unavailable, all AI features fail.	Implement retry with exponential backoff. Add graceful degradation message for users. Future: secondary model fallback.

Lazada API	Live competitor product data for pricing strategy feature.	High, API changes or rate limits break the pricing feature.	Redis caching reduces call frequency. Mock data mode available for development. Graceful error stream on failure.
Resend API	Transactional email delivery of generated reports.	Medium, email delivery failure does not break report generation; files are still created.	Log failure. Return status to user. Future: retry queue.
ChromaDB	Persistent vector storage for all business data.	Medium, local disk dependency. Corruption or disk full would break retrieval.	Regular backup of ./chroma_db directory recommended.
Redis	In-memory cache for pricing strategy results.	Medium, if unavailable, all pricing requests hit Lazada live (increased cost + rate limit risk).	Document as hard dependency. Health check on startup. Future: Redis Sentinel for HA.

Project Management & Team Contributions

Project Timeline

Day(s)	Phase	Activities
Day 1 (Apr 16)	Kickoff & Planning	Domain reveal attendance, problem statement analysis, LoCor AI concept finalisation, technology stack confirmation (React 19 + FastAPI + ChromaDB + Redis + Z.AI GLM-4.5-Flash), GitHub repository setup, task assignment per team member role
Day 2–3 (Apr 17–18)	Architecture & Design	System architecture design (dependency diagram, sequence diagram), ChromaDB schema design, SQLite chat history schema, FastAPI endpoint planning, React page structure (Upload/Dashboard/Insights/Chat), TOON encoding design, Z.AI GLM prompt engineering planning
Day 4–5 (Apr 19–20)	Core Backend Development	FastAPI server.py setup, processing_tools/parser.py (PyMuPDF + python-docx + pandas), db/data_feeder.py ChromaDB ingestion, db/query.py vector search, chat_history/sql.py SQLite setup, data_apis/ competitor API integrations and Redis caching
Day 6–7 (Apr 21–22)	GLM Integration & Agentic Loop	ai.py RAG chat and insights generation, TOON encoding implementation, Z.AI GLM-4.5-Flash integration, main.py agentic orchestration loop, processing_generation/tools.py tool definitions, generate.py (.docx/.xlsx), newsletter.py (Resend API)

Day 8 (Apr 23)	Frontend Development & Integration	React 19 (Vite) frontend, Upload.jsx, Dashboard.jsx, Insights.jsx, Chat.jsx, TailwindCSS 4 styling, api/init.js + api/chat.js + api/insights.js JavaScript API modules, FastAPI-frontend integration and end-to-end testing
Day 9 (Apr 24)	Testing & Documentation	End-to-end scenario testing with front_end/sample/ documents, GLM dependency validation, Redis cache testing, PRD/SAD/QA document completion, bug fixing
Day 10 (Apr 25)	Pitch & Submission	Pitch deck creation, 10-minute demo video recording with prototype demonstration, final submission review and upload to umhackathon.com before 07:59:59 on 26 April 2026

Team Member Roles

Name	**Role**	**Responsibilities**
Gilbert Gan Wei Xuan	Team Lead / Backend Developer	ChromaDB Query Processor and Data Retrieval (db/query.py), overall project coordination, pitch deck preparation
Colin Leong Hong Seng	AI / Prompt Engineer & Backend Developer	Fetch e-commerce APIs, API response synthesis by Z.AI, set up Redis cache, UI and backend logic for pricing strategy, README.md documentation.
Gan Jayci	Frontend Developer & QA / Documentation PIC	React.js UI development, PRD writing, SAD writing, QA Testing Document writing, test case design and execution

Ow Ji Joe	Backend Developer	ChromaDB Data Loading (db/data_feeder.py), document ingestion pipeline, pitch deck preparation
Tan Wen Zhe	AI / Prompt Engineer & Backend Developer	Z.AI GLM-4.5-Flash integration, prompt engineering, FastAPI server.py setup, SQLite configuration (chat_history/sql.py), Resend API newsletter integration (newsletter.py), document generation (generate.py), React.js UI contribution, JavaScript async API modules (api/chat.js, api/init.js, api/insights.js), frontend-backend integration

Recommendations

- ChromaDB versioning: Implement a collection-reset mechanism on re-upload to prevent stale data accumulation. Each upload should replace, not append, the relevant collection.
- Structured ingest format: Replace the pandas DataFrame repr storage approach with structured JSON. Store each row as a separate ChromaDB document with metadata fields (product_name, category, price, etc.) to enable more precise retrieval and eliminate the fragile regex parser.
- Redis TTL: Set a 24-hour TTL on all pricing cache entries to ensure competitor data remains current.
- Redis health check: Add a startup check in FastAPI's lifespan handler to verify Redis connectivity and log a warning if unavailable.
- AI output validation: Implement a lightweight post-generation validator for dashboard and insights endpoints that cross-checks AI-generated figures against source data in ChromaDB before returning the response.
- Authentication: For any multi-user deployment, add JWT-based authentication and per-user ChromaDB namespace isolation.
- Cloud deployment: Migrate ChromaDB to a managed vector database (e.g. Pinecone or Weaviate Cloud) and Redis to AWS ElastiCache for production reliability and scalability.

References & Citations

All external tools, libraries, APIs and resources referenced in this document are cited below in IEEE format, in compliance with UMHackathon 2026 submission rules.

#	Citation (IEEE Format)
[1]	Z.AI, "GLM-4.5-Flash," Z.AI. [Online]. Available: https://docs.z.ai/guides/llm/glm-4.5 [Accessed: Apr. 2026].
[2]	Chroma, "ChromaDB," Chroma. [Online]. Available: https://www.trychroma.com. [Accessed: Apr. 2026].
[3]	Redis Ltd., "Redis," Redis. [Online]. Available: https://redis.io. [Accessed: Apr. 2026].
[4]	S. Ramirez, "FastAPI," FastAPI. [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Apr. 2026].
[8]	Artifex Software, "PyMuPDF," PyMuPDF. [Online]. Available: https://pymupdf.readthedocs.io. [Accessed: Apr. 2026].
[9]	python-docx Contributors, "python-docx," python-docx. [Online]. Available: https://python-docx.readthedocs.io. [Accessed: Apr. 2026].
[11]	Resend Inc., "Resend," Resend. [Online]. Available: https://resend.com. [Accessed: Apr. 2026].
[13]	The pandas Development Team, "pandas," pandas. [Online]. Available: https://pandas.pydata.org. [Accessed: Apr. 2026].

AI Tools Used

In compliance with UMHackathon 2026 rules requiring proper citation of all AI tools used, the following tools were referenced during the preparation of this document:

#	Citation (IEEE Format)
[A1]	Z.AI, "GLM-4.5-Flash," Z.AI. [Online]. Available: https://docs.z.ai/guides/llm/glm-4.5 [Accessed: Apr. 2026].

[A2]	Anthropic, "Claude AI Assistant," Anthropic, 2025. [Online]. Available: https://claude.ai. [Accessed: Apr. 2026].

AI Usage

In compliance with UMHackathon 2026 submission requirements, all AI tools used during the development of LoCor AI are declared below.

Z.AI GLM-4.5-Flash is the core reasoning engine built into LoCor AI itself. It handles RAG-powered chat, business insight generation, competitor pricing analysis and automated report generation. This is a product component, not a development aid.

Claude (Anthropic) was referred to for writing ideas and ideation suggestions when preparing the PRD, SAD and QATD. All written content was drafted by our team based on our own understanding of the system. AI was not used to write the final content directly. Diagrams in this document were also generated with the assistance of Claude (Anthropic).

During development, we also referred to AI for guidance in a few specific areas:

1. Frontend: UI component structure and layout ideas for the React 19 pages
2. Prompt engineering: prompt structure and system persona design for the GLM integration
3. JSON mapping: mapping backend API responses to frontend components
4. Code cleanup: code readability and refactoring suggestions

In all of these cases, the ideas, decisions and actual implementation were done by our team. AI was used as a reference, not as a replacement for our own work.